MECHATRONICS AND IOT

ASSIGNMENT REPORT-4

TITLE:

Design and simulate an Air Pollution Monitoring System using MQ2 Gas Sensor. The system detects harmful gases and monitors air quality levels for industrial and environmental safety applications.

TEAM MEMBERS:

KOWSHIK K 24MER025

NITHISH J 24MER037

KAVIN KRISHNA D S - 24MER024

LOGESH S 24MER027

Project Overview:

The Air Pollution Monitoring System is an embedded system designed to detect and monitor harmful gases present in the surrounding air. The system uses an MQ2 gas sensor to detect gases such as smoke, LPG, methane and other combustible gases. An ESP8266 microcontroller collects the sensor readings, processes the data and determines the air-quality level.

The measured gas concentration is compared with predefined threshold values. Based on the detected level, the system can classify the environment as Safe, Moderate, or Hazardous. An alert can be generated when the gas concentration exceeds the safe limit. The system can also be extended with a display or IoT platform for real-time monitoring.

The proposed system is suitable for industrial safety, workshops, laboratories, kitchens, indoor environments and environmental monitoring applications.

Problem Statement:

Air pollution caused by harmful gases and smoke can create serious risks to human health, industrial equipment and environmental safety. In industrial environments, gas leakage or excessive smoke may occur without immediate detection, potentially leading to health hazards, fire accidents or equipment damage.

Conventional monitoring systems can be expensive and may not be suitable for small-scale or low-cost applications. Therefore, there is a need for a low-cost, compact and real-time air pollution monitoring system capable of detecting harmful gases and providing immediate warning when pollution levels become unsafe.

Proposed Solution:

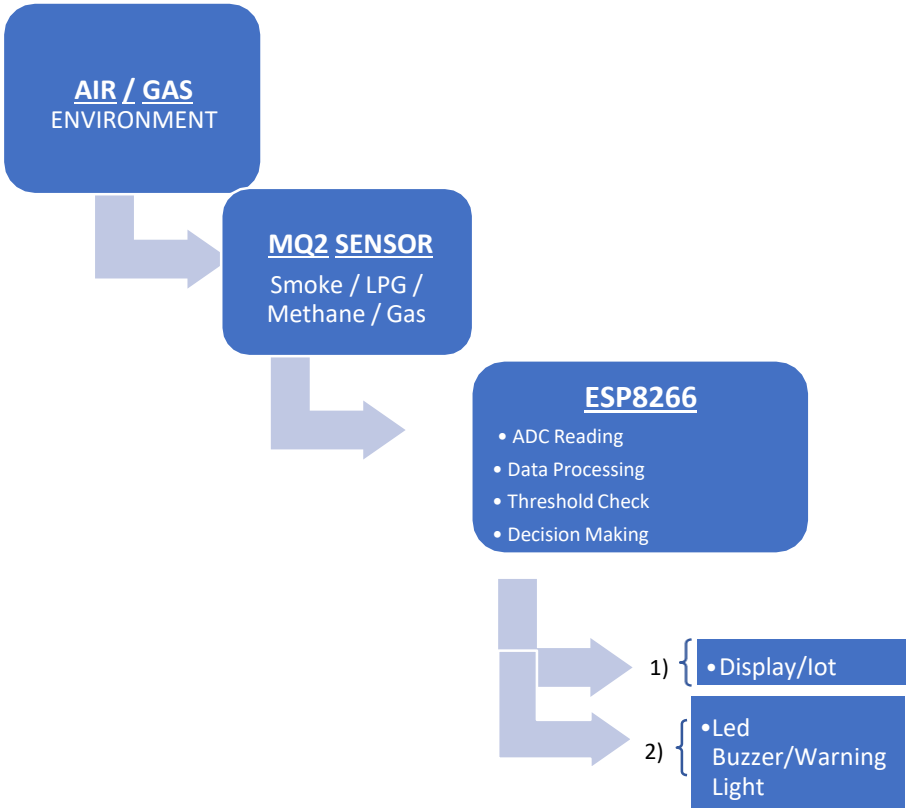
The proposed system uses an MQ2 gas sensor integrated with an ESP8266 microcontroller to continuously monitor the surrounding air.

Working principle:

1. The MQ2 sensor senses smoke and combustible gases in the surrounding environment.
2. The sensor produces an analog output corresponding to the detected gas level.
3. The ESP8266 reads this analog signal through its ADC.
4. The ESP8266 processes the sensor reading and compares it with predefined threshold values.
5. The pollution level is classified into different categories such as:
    - Safe
    - Moderate
    - Hazardous
6. If the detected level exceeds the hazardous threshold, an LED/buzzer alert is activated.
7. The system can optionally send the readings to an IoT/cloud platform through the ESP8266's Wi-Fi capability.

8. The readings can be displayed on a serial monitor, OLED/LCD display or web dashboard.

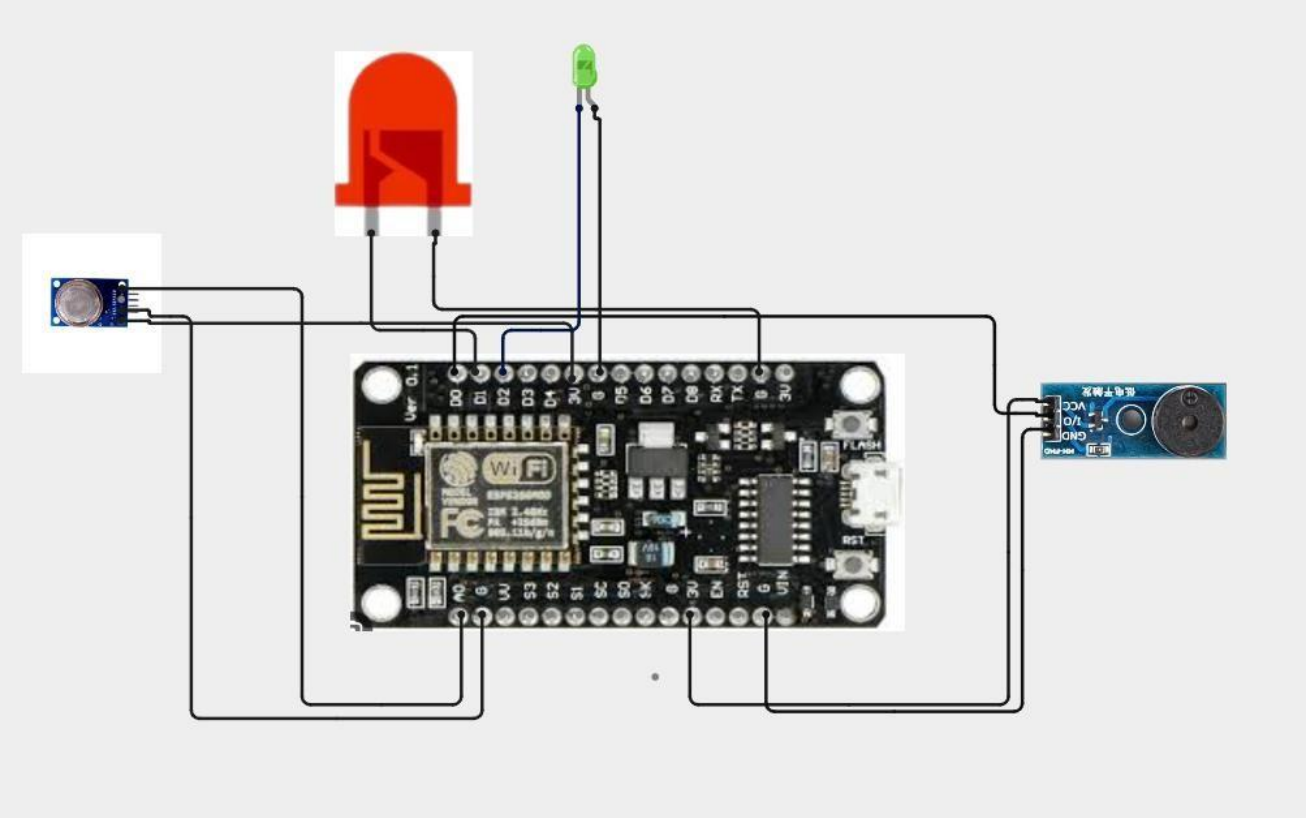
System Architecture:



Circuit Table :

Component	Pin/Terminal	ESP8266 Connection	Purpose
MQ2 Gas Sensor	VCC	5V/IN	Power Supply
Green LED	Anode	GPIO 25 through 220 Ω	Safe indication
Red LED	Anode	GPIO 27 through 220 Ω	Hazardous indication
Buzzer	Positive	GPIO 14	Audible warning

Circuit Design:



Algorithm:

START

↓

Initialize ESP8266

↓

Initialize MQ2 sensor

↓

Initialize LEDs, buzzer and display

↓

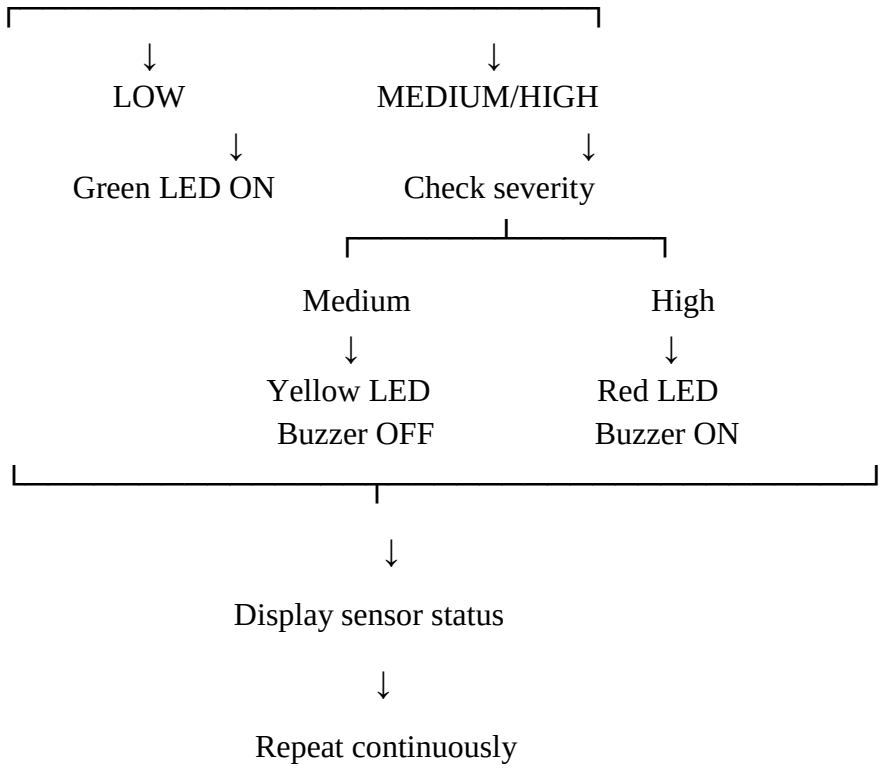
Read analog value from MQ2

↓

Process / average sensor reading

↓

Compare reading with threshold



Coding:

```
 #include <ESP8266WiFi.h>
#include <ThingSpeak.h>
// ================================
// PIN DEFINITIONS
// ================================
#define gassensorPin A0
#define buzzPin 2
#define redlPin 5
#define greenlPin 4
#define DANGER_LEVEL 200
// ================================
// WIFI DETAILS
// ================================
const char* ssid = "Luky003"; const
char* password = "kowshik03";
// ================================
```

```
// THINGSPEAK DETAILS
//  ==============================  unsigned
long channelID = 3455657; const char* writeAPIKey =
"NAE0K4GXMAW2BMJ7";
WiFiClient client;
// ==============================
// TIMER
// ==============================
unsigned long previousMillis = 0; const unsigned
long thingSpeakInterval = 60000;
// ==============================
// SETUP
// ==============================
void setup()
{
  Serial.begin(9600);
  //                Pin                setup
  pinMode(gassensorPin,
  INPUT);        pinMode(buzzPin,
  OUTPUT);      pinMode(redlPin,
  OUTPUT);  pinMode(greenlPin,
  OUTPUT);
  digitalWrite(buzzPin,      LOW);
  digitalWrite(redlPin,       LOW);
  digitalWrite(greenlPin, LOW);
  // ==============================
  // WIFI CONNECTION
  // ==============================
  WiFi.mode(WIFI_STA);
  WiFi.begin(ssid, password);
  Serial.println();
```

```
  Serial.print("Connecting to WiFi"); int
  attempts = 0;
  while (WiFi.status() != WL_CONNECTED && attempts < 40)
  {
    delay(500);
    Serial.print(".");
    attempts++;
  }
  Serial.println();
  // ==============================
  // CHECK WIFI
  //   ==============================
  if (WiFi.status() == WL_CONNECTED)
  {
    Serial.println("WiFi Connected!");
    Serial.print("IP Address: ");
    Serial.println(WiFi.localIP());
    Serial.print("Signal Strength: ");
    Serial.print(WiFi.RSSI());
    Serial.println(" dBm");
    // Start ThingSpeak
    ThingSpeak.begin(client);
  }
  else
  {
    Serial.println("WiFi Connection Failed!");
    Serial.println("Check WiFi name and password.");
  }
}
// ==============================
// LOOP
```

```
// ================================
void loop()
{
  // ================================
  // READ GAS SENSOR
  // ================================
  int gasValue = analogRead(gassensorPin);
  // ================================
  // SAFE / DANGER
  // ================================
  int dangerStatus;
  if (gasValue > DANGER_LEVEL)
  {
    Serial.print("Gas Value: ");
    Serial.print(gasValue);
    Serial.println(" // Danger");
    digitalWrite(redlPin,    HIGH);
    digitalWrite(greenlPin,
    LOW);   digitalWrite(buzzPin,
    HIGH); dangerStatus = 1;
  }
  else
  {
    Serial.print("Gas Value: ");
    Serial.print(gasValue);
    Serial.println("      //      Safe");
    digitalWrite(redlPin,     LOW);
    digitalWrite(greenlPin,
    HIGH);   digitalWrite(buzzPin,
    LOW); dangerStatus = 0;
  }
```

```
// ================================
// SEND TO THINGSPEAK EVERY 60 SEC
// ================================
unsigned long currentMillis = millis(); if (currentMillis
- previousMillis >= thingSpeakInterval)
{ previousMillis = currentMillis; if
  (WiFi.status() == WL_CONNECTED)
  {
    Serial.println("                                    ");
    Serial.println("Sending data to ThingSpeak...");
    // Field 1 = Gas Value
    ThingSpeak.setField(1, gasValue);
    // Field 2 = Status
    // 0 = Safe
    // 1 = Danger
    ThingSpeak.setField(2, dangerStatus);
    int response = ThingSpeak.writeFields(channelID, writeAPIKey); if
    (response == 200)
    {
      Serial.println("ThingSpeak: Data sent successfully!");
    }
    else
    {
      Serial.print("ThingSpeak Error: ");
      Serial.println(response);
    }
    Serial.println("                                    ");
  }
  else
  {
```

```
    Serial.println("WiFi
 disconnected!"); } } delay(500);
}
```

System Working:

- The system begins by powering the MQ2 gas sensor and ESP8266. The MQ2 sensor continuously senses the surrounding atmosphere for smoke and combustible gases such as LPG and methane.
- The sensor generates an analog signal based on the detected gas concentration. This signal is supplied to the ESP8266 ADC through GPIO 34. The ESP32 reads and processes the sensor value and compares it with predefined threshold levels.
- When the sensor reading is below the safe threshold, the system considers the environment to be Safe, and the green LED is activated. When the reading reaches the moderate range, the yellow LED indicates that the gas level requires attention.
- If the sensor reading exceeds the hazardous threshold, the red LED and buzzer are activated to provide an immediate warning. The sensor value and status can also be displayed through the Serial Monitor or an optional OLED display.
- The ESP8266 continuously repeats this process, allowing the system to provide realtime gas and smoke monitoring. Its Wi-Fi capability can further be used to transmit the readings to an IoT dashboard for remote monitoring.

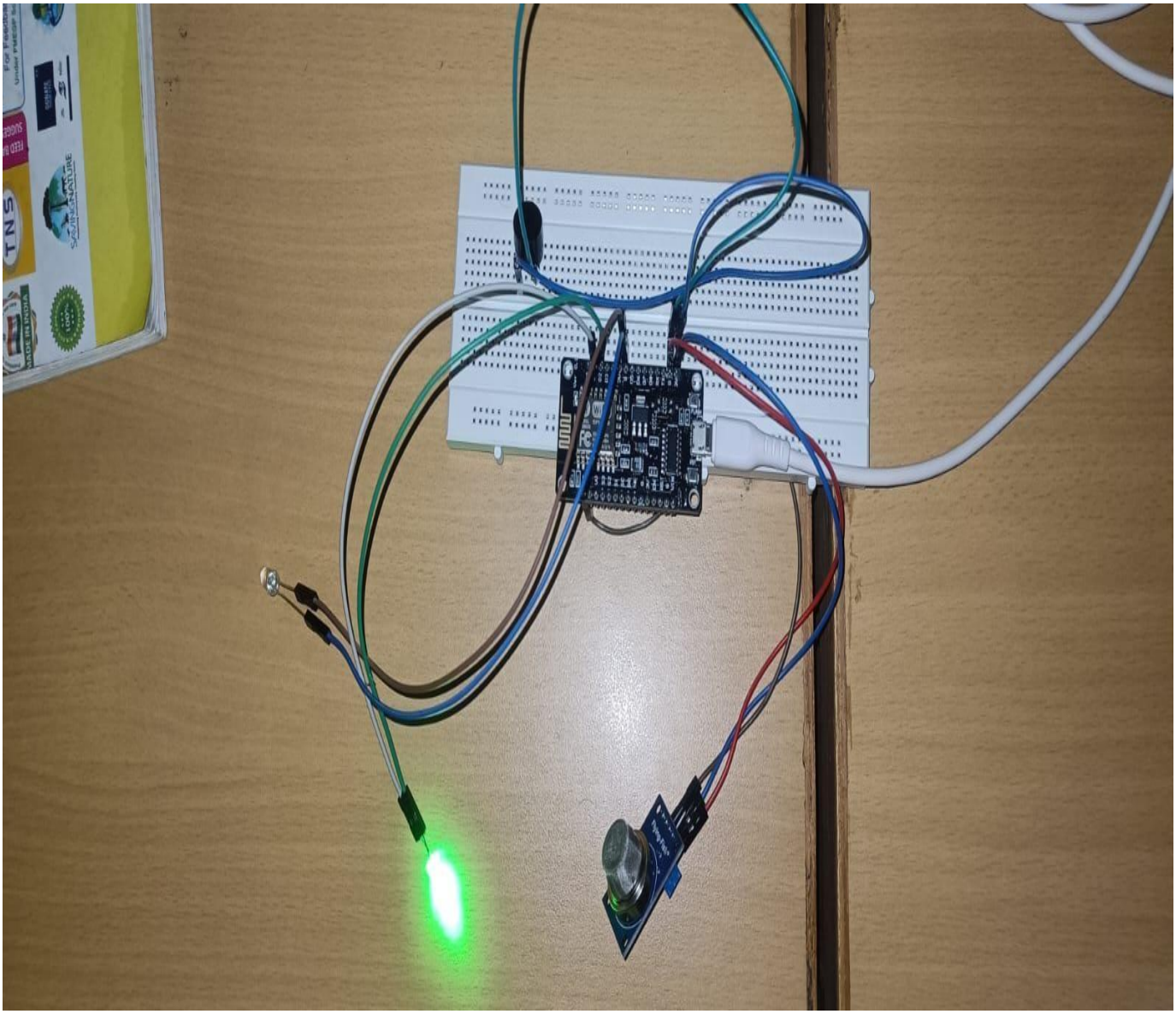
Bill Of Material:

S. no	Component	Specification	Quantity	Cost
1	ESP8266	Node MCU ESP8266	1	Rs. 450
2	MQ2 Gas Sensor	Smoke/LPG/Methane detectio	1	Rs. 100
3	Green LED	5 mm	1	Rs. 20
4	Buzzer	Active Buzzer	1	Rs. 40
5	Breadboard	Standard 830-point	1	Rs. 100
6	Jumper Wire	Male-Male/Male-Female	1 set	Rs. 50

			Total	Rs. 760

Prototype Implementation:

- The prototype of the Air Pollution Monitoring System is implemented using a NodeMCU ESP8266, an MQ2 gas sensor, LEDs, and a buzzer. The ESP8266 acts as the main controller and continuously processes the gas sensor readings.
- The MQ2 sensor is connected to the analog input A0 of the ESP8266 through a suitable voltage divider. The sensor detects smoke and combustible gases such as LPG and methane and produces an analog output corresponding to the detected gas level.



- Three LEDs are used to indicate the pollution condition. The green LED indicates a safe condition, the yellow LED indicates a moderate gas level, and the red LED indicates a hazardous gas level. An active buzzer is activated when the gas level exceeds the hazardous threshold.
- The ESP8266 continuously reads the MQ2 sensor output, compares it with predefined threshold values and activates the corresponding indicators. Since the ESP8266 has built-in Wi-Fi connectivity, the prototype can also be extended to transmit sensor readings to an IoT platform or web dashboard for remote monitoring.

Results and Performance:

- The developed Air Pollution Monitoring System using MQ2 Gas Sensor and ESP8266 was successfully implemented and tested under different gas and smoke conditions. The system continuously monitored the gas sensor output and classified the detected level into Safe, Moderate, and Hazardous conditions.
- The prototype demonstrated that the MQ2 sensor can effectively detect changes in the surrounding gas/smoke level, while the ESP8266 successfully processed the sensor output and generated appropriate warning indications.

The system provided:

- Real-time monitoring of gas/smoke levels.
- Fast response when the sensor detected an increase in gas concentration.
- Three-level indication using green, yellow and red LEDs.